Computer Networks

5. Network Layer: Control Plane 2

Network Layer - Control Plane: Overview

- Introduction
- Routing protocols
  - link state
  - distance vector
- Intra-ISP routing: OSPF
- Routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol

거리 벡터 알고리즘 (distance vector algorithm)

벨만-포드 (Bellman-Ford) 공식을 기반 (dynamic programming):

Bellman-Ford equation

Let $D_x(y)$: cost of least-cost path from x to y.
Then:

$$D_x(y) = \min_v \{ c_{x,v} + D_v(y) \}$$

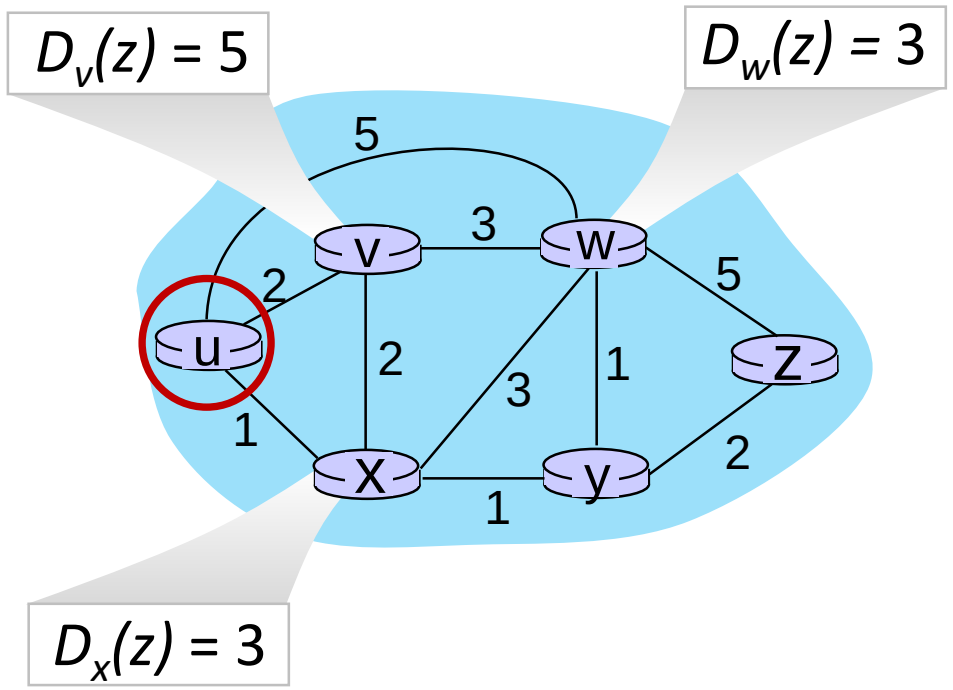
min taken over all neighbors v of x

direct cost of link from x to v

v's estimated least-cost-path cost to y

Bellman-Ford Example

*u*의 이웃 노드 *x*, *v*, *w*는 목적지 노드 *z*까지 추정된 최소 비용 경로의 비용을 알고 있다고 가정:



Bellman-Ford equation says:

$$
\begin{aligned}
D_u(z) &= \min \{ c_{u,v} + D_v(z), \\
&\qquad c_{u,x} + D_x(z), \\
&\qquad c_{u,w} + D_w(z) \} \\
&= \min \{2 + 5, \\
&\qquad 1 + 3, \\
&\qquad 5 + 3\} = 4
\end{aligned}
$$

최소 비용을 달성하는 노드 *x*는 목적지 노드 *z*까지 최소 비용 경로 상의 다음 홉 (hop) 됨

Distance vector algorithm

node *x* initially:

- knows cost to each neighbor *v*: c(x, v)

key idea:

- 자신의 거리 벡터 (DV) 추정 값이 갱신되면, 각 노드는 자신의 새로운 DV 추정 값을 이웃 노드들에게 전송
- 노드 *x*가 이웃 노드로부터 새로운 DV 추정 값을 수신하면, 벨만-포드 방정식을 사용하여 자신의 DV를 갱신:

$$D_x(y) \leftarrow min_v\{c_{x,v} + D_v(y)\} \text{ for each node } y \in N$$

- $D_x(y)$의 추정 값은 실제 최소 비용 $D_x(y)$에 점차 수렴하게 됨

Distance vector algorithm

각 노드:

대기 (링크의 비용이 변경되거나 이웃 노드부터 알림을 받을 때까지)

변경된 링크 비용 또는 이웃 노드부터 받은 DV를 사용하여 DV 추정 값을 재계산

어떤 목적지에 대한 DV 추정 값이 변경된다면, 이웃에게 알림

반복적, 비동기적: 각 노드의 반복 계산은 다음 경우에 수행됨

- 로컬 링크 비용이 변경된 경우
- 이웃 노드로부터 DV 갱신 메시지를 받은 경우

분산적, 자기 종료: 각 노드는 자신의 DV가 변경될 때만 이웃 노드에게 알림

- 이웃 노드의 DV가 변경되면, 해당 이웃 노드는 다시 자신의 이웃 노드에게 알림
- 알림을 받지 않은 노드는 추가 동작을 수행하지 않음

Distance vector: example

$D_x()$

from \ cost to	x	y	z
x	0	2	7
y	∞	∞	∞
z	∞	∞	∞

$D_y()$

from \ cost to	x	y	z
x	∞	∞	∞
y	2	0	1
z	∞	∞	∞

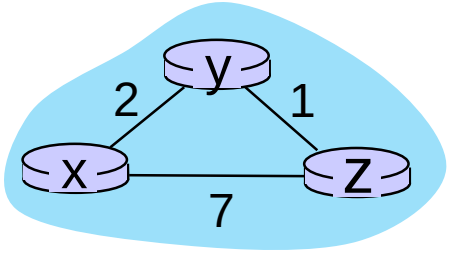
$D_z()$

from \ cost to	x	y	z
x	∞	∞	∞
y	∞	∞	∞
z	7	1	0

from \ cost to	x	y	z
x	0	2	3
y	2	0	1
z	7	1	0

$D_x(z) = \min\{c_{x,y} + D_y(z),\ c_{x,z} + D_z(z)\}$
$= \min\{2+1,\ 7+0\} = 3$

$D_x(y) = \min\{c_{x,y} + D_y(y),\ c_{x,z} + D_z(y)\}$
$= \min\{2+0,\ 7+1\} = 2$



time

Distance vector: example

$D_x()$

from \ cost to	x	y	z
x	0	2	7
y	∞	∞	∞
z	∞	∞	∞

$D_y()$

from \ cost to	x	y	z
x	∞	∞	∞
y	2	0	1
z	∞	∞	∞

$D_z()$

from \ cost to	x	y	z
x	∞	∞	∞
y	∞	∞	∞
z	7	1	0

from \ cost to	x	y	z
x	0	2	3
y	2	0	1
z	7	1	0

from \ cost to	x	y	z
x	0	2	7
y	2	0	1
z	7	1	0

from \ cost to	x	y	z
x	0	2	7
y	2	0	1
z	3	1	0

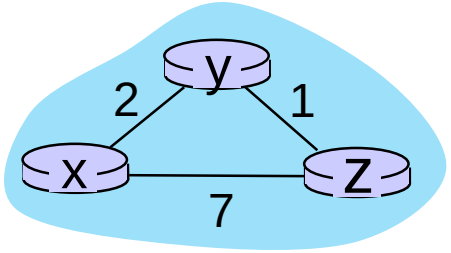
from \ cost to	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0

from \ cost to	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0

from \ cost to	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0

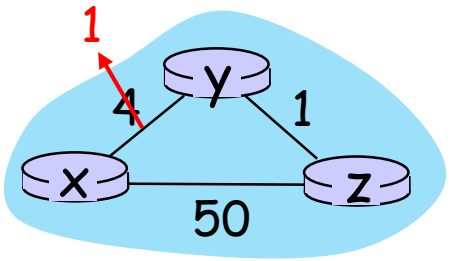
from \ cost to	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0

time



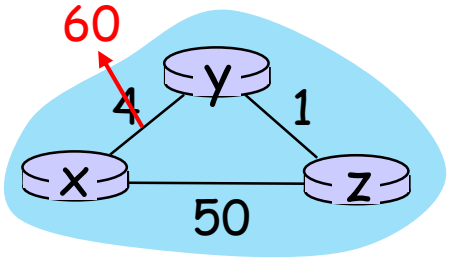
Distance vector: link cost changes

링크 비용 변경: ‘good news travels fast’



- 노드는 자신의 링크 비용 변경을 감지
- updates routing info, recalculates local DV
- if DV changes, notify neighbors:
  - t_1: *y*가 링크 비용의 변화 (4→1)를 감지하고, 자기의 거리 벡터를 갱신 후 이 변경 값을 이웃에게 알림
  - t_2: *z*는 *y*로부터 갱신 정보를 받고 자신의 테이블을 갱신함. *z*는 *x*까지의 새로운 최소 비용을 계산 (5→2)하고, 이웃에게 자신의 새로운 거리 벡터를 알림
  - t_3: *y*는 *z*로 부터 갱신 정보를 받고 자신의 테이블을 갱신함. *y*의 최소 비용은 변화가 없으므로 *y*는 *z*에게 아무런 메시지를 보내지 않고, 정지 (수렴) 상태가 됨

Distance vector: link cost changes



링크 비용 변경: ‘bad news travels slow’

- 노드는 자신의 링크 비용 변경을 감지
- 무한 계수 문제 (count-to-infinity problem):
  - *y*는 *x*의 직접 링크 비용이 60으로 변경된 것을 탐지하지만, 이전에 *z*가 *y*에게 *x*로 가는 비용이 5인 경로가 있다고 알렸음. 따라서 *y*는 *z*를 통해 *x*로 가는 경로의 비용이 6이라고 새롭게 계산하고, *z*에게 *x*로 가는 비용이 6이라고 알림
  - *z*는 *y*를 통해 *x*로 가는 경로의 비용이 6인 것을 알게 되므로, *z*는 *y*를 통해 *x*로 가는 경로의 비용을 7로 새롭게 계산하고, *y*에게 *x*로 가는 경로의 비용은 7이라고 알림
  - *y*는 *z*를 통해 *x*로 가는 경로의 비용이 7인 것을 알게 되므로, *y*는 *z*를 통해 *x*로 가는 경로의 비용을 8로 새롭게 계산하고, *z*에게 *x*로 가는 경로의 비용은 8이라고 알림
  - *z*는 *y*를 통해 *x*로 가는 경로의 비용이 8인 것을 알게 되므로, *z*는 *y*를 통해 *x*로 가는 경로의 비용을 9로 새롭게 계산하고, *y*에게 *x*로 가는 경로의 비용은 9이라고 알림
- 포이즌 리버스 (poisoned reverse):
  - 만약 *z*가 *x*에 도달하기 위해 *y*를 경유하는 경우: *z*는 *y*에게 *x*까지의 경로의 비용이 ‘무한’이라고 알림 → *y*는 *x*로 가기 위해서 *z*를 경유하지 않음

Comparison of LS and DV algorithms

링크 상태 (Link state)

- 라우터는 해당 영역의 전체 네트워크 토폴로지 (그래프)를 알고 있음
- 정보를 갱신하기 위해 전체 네트워크에 플러딩, O(|N||E|)개 메시지 전송
- 알고리즘 결과의 수렴 속도가 빠름
- 경로 진동 문제
- 대규모 네트워크에서 설정이 복잡함
- 라우터 오작동/장애 시 잘못된 링크 정보가 플러딩
  - 각 라우터는 자신의 포워딩 테이블만 계산 (계산 결과는 전파하지 않음)
- [예] OSPF

거리 벡터 (distance vector)

- 라우터는 전체 네트워크 토폴로지를 알지 못함
- 정보를 갱신하기 위해 직접 연결된 이웃에게만 메시지를 전송
- 알고리즘 결과의 수렴속도가 느림
- 라우팅 루프, 무한 계수 문제
- 상대적으로 간단함
- 각 라우터의 거리 벡터를 다른 라우터도 사용: 오류가 네트워크 전체로 확산 가능
  - '모든 목적지로 가는 최저 비용의 경로가 있다!': 블랙홀 (black-holing) 공격
- [예] RIP

Network Layer - Control Plane: Overview

- Introduction
- Routing protocols
  - link state
  - distance vector
- Intra-ISP routing: OSPF
- Routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol

Making routing scalable

지금까지 이상적인 라우팅 환경을 가정함

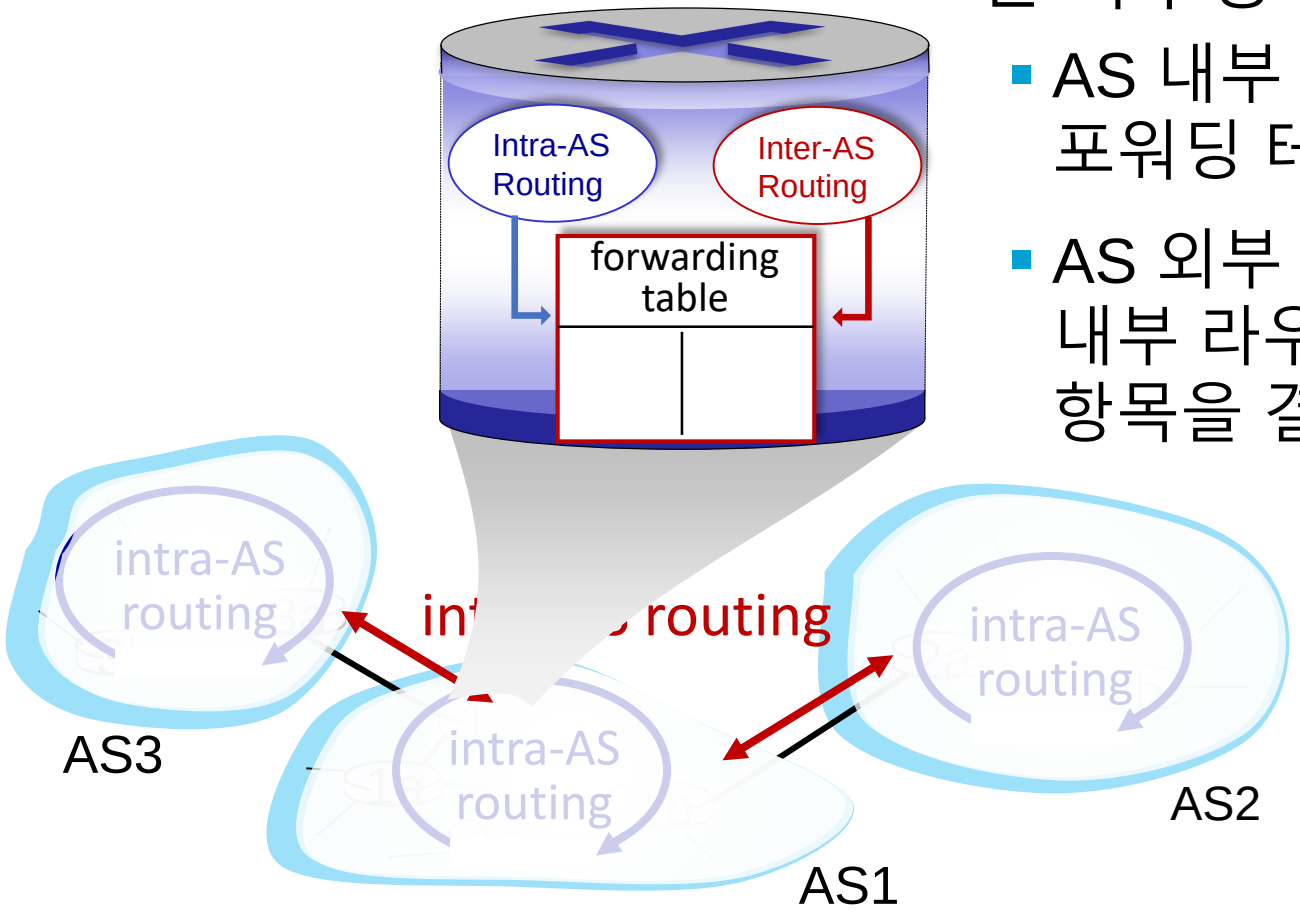
- 모든 라우터가 동일
- 모든 라우터가 같은 네트워크 평면에 존재 (즉, 계층적 구조를 가지고 있지 않음)
  - …하지만 현실은 그렇지 않음!
- 확장성 문제 (scalability issue): 실제로 수십억 개의 목적지가 존재
  - 라우팅 테이블에 모든 목적지를 저장할 수 없음
  - 모든 라우터 사이에서 연결 상태와 링크 비용을 플러딩한다면 엄청난 오버헤드가 발생
- 관리 자율성 (administrative autonomy):
  - Internet: a network of networks
  - 각 네트워크 관리자 (ISP)는 자신의 네트워크를 자신이 원하는 대로 운용/관리하고 싶음

Internet approach to scalable routing

라우터들을 자율 시스템 (autonomous systems, AS, domain)으로 조직화하여 문제를 해결

- 각 AS는 동일한 관리주체에 의해 동일한 정책으로 관리되는 라우터들과 서브넷들로 구성
  - 장점: 라우팅 정책의 독립성, 보안 유지, 트래픽 감소, 오류 국지화
- intra-AS (intra-domain): routing among within same AS (‘network’)
  - 같은 AS에 있는 라우터들은 동일한 도메인 라우팅 프로토콜을 실행
  - 다른 AS에 있는 라우터들은 서로 다른 도메인 라우팅 프로토콜을 실행할 수 있음
  - gateway router (border router): AS의 경계에 있으며, 다른 AS에 속한 라우터와 링크로 연결됨
- inter-AS (inter-domain): routing among AS’es
  - 게이트웨이 라우터는 도메인 간 라우팅 (및 도메인 내 라우팅)을 수행

Interconnected ASes



포워딩 테이블은 AS 내부 라우팅과 AS 간 라우팅 알고리즘을 통해 구성됨

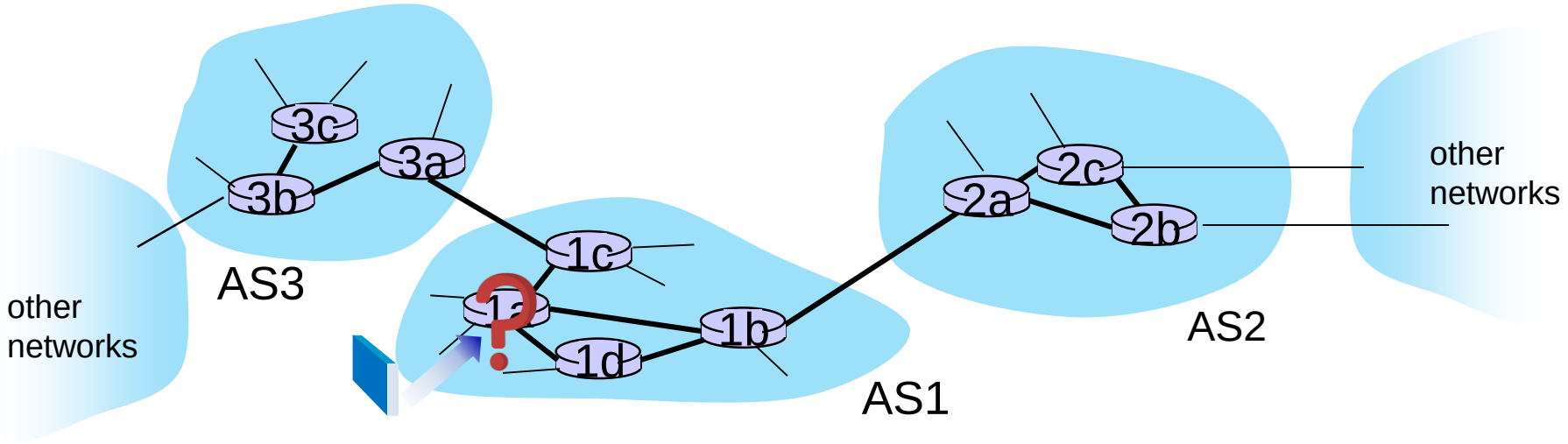
- AS 내부 목적지: AS 내부 라우팅이 포워딩 테이블 항목 (entry)을 결정
- AS 외부 목적지: AS 간 라우팅과 AS 내부 라우팅이 함께 포워딩 테이블 항목을 결정

Inter-AS routing: a role in intra-domain forwarding

- AS1의 라우터가 AS1 외부로 향하는 데이터그램을 수신했다고 가정:
  - 해당 라우터는 패킷을 AS1의 어느 게이트웨이 라우터로 전달?

AS1 inter-domain routing must:

- ① AS2를 통해 도달할 수 있는 목적지와 AS3를 통해 도달할 수 있는 목적지를 파악
- ② 해당 도달 가능성 정보를 AS1의 모든 라우터에 전파



Intra-AS routing: routing within an AS

most common intra-AS routing protocols:

- RIP: Routing Information Protocol [RFC 1723]
  - classic DV: DVs exchanged every 30 secs
  - no longer widely used
- EIGRP: Enhanced Interior Gateway Routing Protocol
  - DV 기반
  - formerly Cisco-proprietary for decades (became open in 2013 [RFC 7868])
- OSPF: Open Shortest Path First (개방형 최단 경로 우선) [RFC 2328]
  - 링크 상태 라우팅
  - IS-IS 프로토콜 (RFC 표준이 아닌 ISO 표준)은 OSPF와 본질적으로 동일

OSPF (Open Shortest Path First) routing

- 가장 널리 사용되는 intra-AS 라우팅 프로토콜
- ‘Open’: 공개 표준으로 누구나 구현 가능
- 링크 상태 라우팅 알고리즘 기반 (Dijkstra 사용)
  - 각 라우터는 자신이 속한 영역의 토폴로지를 바탕으로 포워딩 테이블을 계산
  - 각 라우터는 자신의 링크 상태를 같은 영역 내부에 flooding 방식으로 전달
  - OSPF는 IP 바로 위에서 동작하며, TCP/UDP를 사용하지 않고 IP 프로토콜 번호 89로 직접 캡슐화됨
  - 동일 비용 다중 경로: 하나의 목적지에 대해 동일한 비용의 경로가 여러 개 있으면, 여러 경로를 통해 트래픽 분산 가능
- 보안: OSPF는 메시지 인증 기능을 지원하여 허가되지 않은 라우터의 라우팅 정보 주입을 방지

계층화된 OSPF

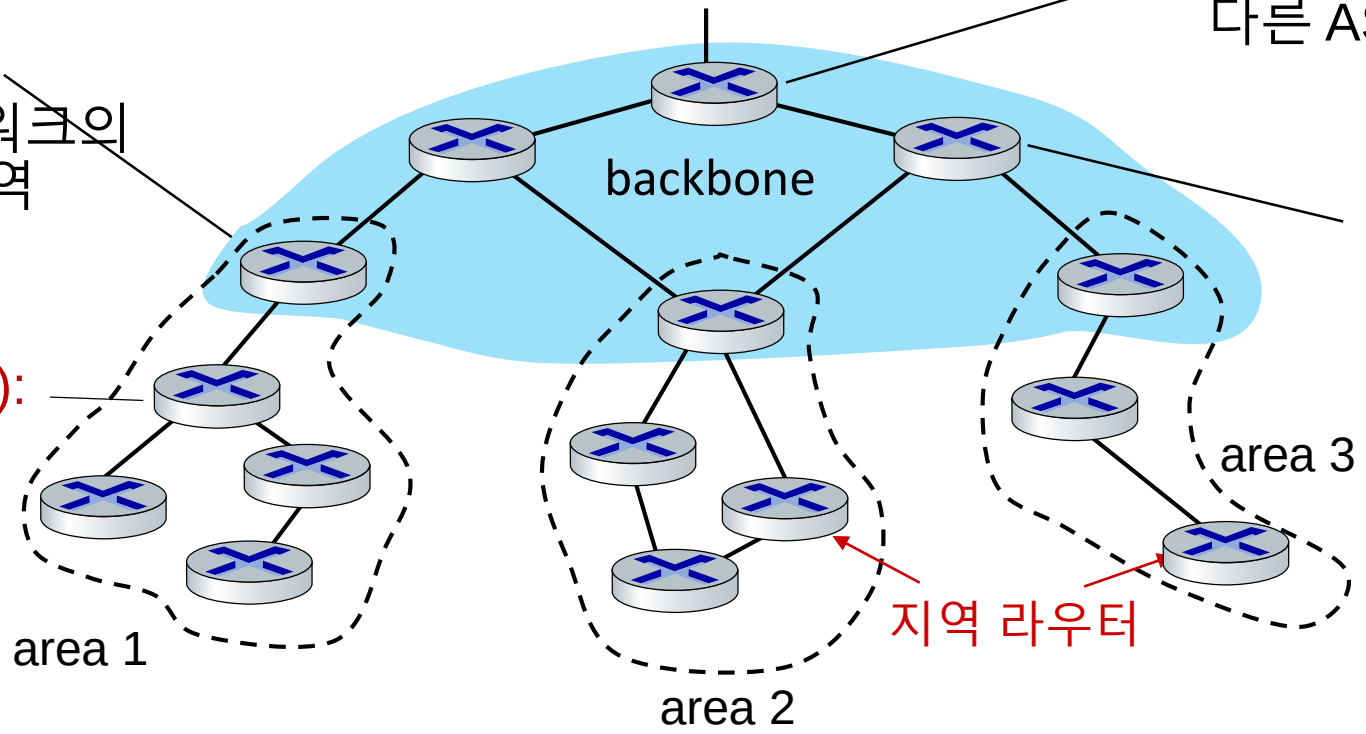
- 2단계 계층 구조: 네트워크를 여러 영역 (area)으로 분할하고, 백본 영역 (backbone area, Area 0)을 통해 연결
  - LSA 메시지는 자신이 속한 영역 내부에서만 플러딩됨
  - 각 라우터는 자신의 영역에 대한 상세한 토폴로지를 알고 있으며, 다른 영역의 네트워크에 대해서는 도달 방향만 알고 있음

AS 경계 라우터 (ASBR): 다른 AS에 연결됨

영역 경계 라우터 (ABR): 자신의 영역에 있는 네트워크의 요약 경로 정보를 다른 영역 경계 라우터에게 알림

백본 라우터 (backbone router): 백본 영역에 한정하여 OSPF 라우팅을 수행

지역 라우터 (local routers):
- LSA는 영역 내부에서만 플러딩
- 영역 내부 라우팅 계산
- 영역 경계 라우터를 통해 외부로 패킷 전달



Q & A